ITA0605 MACHINE LEARNING

ASSIGNMENT 1

Design, Analyse and Validate an Intelligent Early-Warning Decision Support System for Academic Risk Identification Using Concept Learning and Decision Tree Learning

Course Code: ITA0605

Course Name: Machine Learning

Student Name: Thikkavarapu Sanhith

Register Number: 192311228

Department : CSE

Faculty Name: DR Revathi B

TASK 1 – IDENTIFY AND FORMULATE THE ML PROBLEM

1.1 Formal learning problem: Task, Performance, Experience

Task (T): Given a student record available after Week 6, predict whether the student will be classified as AtRisk = Yes or AtRisk = No by the academic council at the end of the semester. The output is an early-warning classification, not a final academic decision.

Performance measure (P): The primary measure is recall for AtRisk = Yes, defined as TP / (TP + FN). The university should also monitor precision, F1-score, specificity, false-positive rate and the confusion matrix. Recall is emphasised because missing a student who needs support is more harmful than flagging a student who later turns out to be safe.

Experience (E): The labelled historical records supplied by the university form the initial experience. Dataset A contains five labelled examples for concept learning. Dataset B contains fourteen labelled examples for decision-tree learning. Test Set T contains six held-out examples and must not be used to select the original tree. In deployment, new Week-6 records and their eventual audited outcomes become additional experience.

1.2 Attributes and their relevance

Attribute	Type and values	Role and relevance	Caution
Attendance	Ordinal categorical: Low, Medium, High	Captures class participation and exposure to teaching. Low attendance can indicate missed instruction and missed assessments.	Attendance can be affected by illness, disability, transport, caring duties or online-learning access.
Assignments	Binary categorical: Irregular, Regular	Regular submission is a direct indicator of keeping up with coursework and deadlines.	Submission timing alone does not measure quality or understanding.
StudyHours	Binary categorical: Low, High	Approximate weekly study effort; low reported hours may indicate insufficient preparation.	Self-reported hours can be inaccurate and may penalise students with different study needs.
PriorCGPA	Binary categorical: Weak, Strong	Summarises prior academic performance and provides a baseline signal of academic preparedness.	Can carry forward earlier disadvantage; it should not be treated as destiny.

Attribute	Type and values	Role and relevance	Caution
LMS	Binary categorical: Passive, Active	Measures engagement with the learning-management system and access to digital course activity.	Offline study or limited internet access can make an engaged student appear passive.
Mentoring	Binary categorical: No, Yes	Records whether a support intervention is already in place and may help describe support needs.	Potentially risky predictor: mentoring may be assigned because an advisor already suspects risk. It can therefore encode prior decisions or intervention access rather than independent student risk.

The most cautious attribute is Mentoring. It is only valid as a predictor if its value is known at the Week-6 decision time. If mentoring is started after an advisor has identified a student as at risk, it becomes a post-decision variable and may leak information about the outcome. The model should record the time at which mentoring status was measured and should audit whether access to mentoring differs across student groups.

1.3 Target concept and instance-space calculations

$c : X \rightarrow \{0, 1\}, \quad c(x) = 1$ *when AtRisk = Yes; otherwise* $c(x) = 0$.

The target is a Boolean function over the Cartesian product of the categorical attribute domains. The function is not specified in advance; it is the unknown relationship to be learned from labelled examples.

Dataset	Attribute cardinalities	Instance-space size	Training examples	Coverage
A	3 × 2 × 2 × 2 × 2 × 2	96	5 distinct examples	5 / 96 = 5.2083%
B	3 × 2 × 2 × 2	24	14 distinct examples	14 / 24 = 58.3333%

$|X_A| = 3 \times 2 \times 2 \times 2 \times 2 \times 2 = 96; \quad |X_B| = 3 \times 2 \times 2 \times 2 = 24.$

Dataset A samples only about one twentieth of its possible combinations. Its learned concept must therefore be treated as highly uncertain outside the observed examples. Dataset B covers a larger fraction of its smaller four-attribute instance space, but it still leaves ten of the twenty-four combinations unseen.

1.4 Learning paradigm

The core task is supervised learning. Every supplied training record includes input attributes and an outcome label AtRisk. The model is asked to learn the mapping from known inputs to the labelled outcome. Unsupervised learning can help discover student profiles, and reinforcement learning could help optimise interventions later, but neither replaces the labelled classification task.

1.5 Machine-learning issues and mitigations

Issue	Evidence in this problem	Mitigation
Small sample size	Dataset A has 5 examples and Dataset B has 14. Many possible combinations are absent.	Use more semesters of data, cross-validation, confidence intervals and conservative pruning.
Label noise	Labels are assigned at semester end and may differ between advisors; Day-4 estimates 5–10% inconsistency.	Audit a sample of labels, adjudicate disagreements and use noise-tolerant models.
Sampling bias	Historical data may overrepresent students who were already referred to advisors or students with better LMS access.	Compare the training cohort with the full student population and stratify evaluation by relevant groups.
Data leakage	Mentoring may be assigned after a risk concern has already been identified.	Use only information available at Week 6 and exclude post-decision variables.
Fairness and proxy effects	Attendance and LMS activity can reflect access barriers, health or personal responsibilities.	Audit error rates across groups, review high-impact features and keep a human decision-maker.
Privacy	Student records are sensitive educational data.	Restrict access, minimise collected fields, encrypt records and define retention rules.

Issue	Evidence in this problem	Mitigation
Concept drift	Online study introduces Very Low attendance and new combinations.	Monitor new values and data drift, update domains and retrain on recent audited records.

1.6 Cost of errors and chosen measure

A false negative (FN) is more costly than a false positive (FP). A false negative may leave a student without counselling, mentoring or remedial support until failure becomes visible. A false positive can consume advisor time and may cause unnecessary concern, but it can be reduced through a short human review. Therefore, recall for AtRisk = Yes is the main operational measure. Precision must still be monitored so that advisors are not overwhelmed by too many incorrect flags.

TASK 2 – ANALYSE THE LEARNING PARADIGM (3 MARKS)

2.1 Scenario-specific comparison matrix

Criterion	Supervised learning	Unsupervised learning	Reinforcement learning
Labels available?	Yes. Dataset A and Dataset B contain AtRisk labels.	No outcome labels are required for the clustering step.	Not initially. It needs action outcomes or a reward signal over time.
Output	Predicted AtRisk class or risk score for each student.	Clusters or behavioural profiles; no direct AtRisk class.	A policy for selecting an intervention or action.
Feedback signal	Difference between predicted class and the council-assigned label.	Similarity / density structure in the input records.	Reward after an action, such as improved retention or course completion.
Data needed	Week-6 attributes paired with audited end-of-semester outcomes.	Student feature records; labels are optional.	States, actions, transitions and measurable rewards collected over repeated decisions.
Evaluation	Recall, precision, F1, confusion matrix and calibration on held-out outcomes.	Cluster stability, usefulness to advisors and whether groups are actionable.	Cumulative reward, retention improvement and intervention cost, ideally in a safe trial.
Advisor interpretability	High with a small decision tree or explicit rules.	Profiles can be described, but cluster names do not explain a risk label.	A policy may be difficult to explain and can depend on long-term trade-offs.
Main risk	Overfitting, biased or noisy labels, false negatives.	Clusters may not align with academic risk and can be unstable.	Unsafe exploration, delayed rewards and confounding in intervention outcomes.

2.2 Concrete unsupervised use

A clustering method could group students into profiles such as (i) low attendance and passive LMS use, (ii) regular attendance but irregular assignments, and (iii) active LMS use with strong prior CGPA. Advisors could use the profiles to plan group mentoring sessions or identify patterns not anticipated by the original attributes. However, clustering cannot by itself say that a particular student will fail or provide a validated AtRisk = Yes probability. A cluster is a similarity group, not an outcome label. The final early-warning classification still needs supervised learning and labelled evaluation.

2.3 Reinforcement-learning follow-up problem

Element	Plausible definition for intervention scheduling
State	Current Week-6 profile, risk score, previous interventions, course load and advisor capacity.
Action	Offer counselling, enrol the student in mentoring, schedule remedial classes, or take no additional action.
Reward	Positive reward for course completion and improved learning progress; negative reward for failure, avoidable delay, excessive intervention cost or student burden.

Reinforcement learning is not appropriate for the initial AtRisk classifier because the supplied data contains only static inputs and end-of-semester labels. It does not contain a sequence of chosen interventions, their transitions or reliable rewards. Exploration could also expose students to unequal or ineffective support. A later intervention policy should be developed only after outcomes are measured and an ethically reviewed evaluation plan exists.

2.4 Core choice, strongest counter-argument and rebuttal

Supervised learning is the appropriate paradigm for the core system because the university has labelled historical outcomes and needs a prediction that can be tested against those outcomes. A decision tree is an especially suitable first model because advisors can follow its decision path.

Strongest argument against supervised learning: the labelled dataset is small and some labels may be inconsistent, so the model may learn advisor habits rather than the real causes of academic risk. Rebuttal: this is a reason to improve the data and validate the model, not to ignore the labels. Start with a transparent baseline, audit labels, report uncertainty, evaluate recall on independent records and retain human review. Do not deploy an unvalidated model as an automatic decision-maker.

2.5 Hybrid use

A realistic hybrid is clustering plus supervised classification. Clustering can reveal behavioural profiles and help advisors design support programmes; the supervised tree still predicts AtRisk from labelled outcomes. Cluster membership can be used for exploratory reporting or as a carefully validated additional feature. The cluster must not be treated as a risk label.

TASK 3 – FORMULATE THE CONCEPT LEARNING TASK (3 MARKS)

3.1 Target concept and labelled examples

c(x) = 1 ⇔ AtRisk(x) = Yes; c(x) = 0 ⇔ AtRisk(x) = No.

Example	Instance in attribute order	Label	Training type
E1	<Low, Irregular, Low, Weak, Passive, No>	Yes	Positive
E2	<Low, Irregular, Low, Strong, Passive, No>	Yes	Positive
E3	<High, Regular, High, Strong, Active, Yes>	No	Negative
E4	<Low, Irregular, High, Weak, Passive, Yes>	Yes	Positive
E5	<Medium, Irregular, High, Strong, Active, No>	No	Negative

3.2 Hypothesis representation

A hypothesis is a conjunction of six attribute constraints. A specific domain value means that only that value is accepted, "?" means any value is accepted, and "∅" means no value is accepted. The most general hypothesis is <?,?,?,?,?,?>. The most specific hypothesis is <∅,∅,∅,∅,∅,∅>. The hypotheses corresponding exactly to E1 and E3 are their six-value tuples shown in the table above.

3.3 Syntactic hypotheses, semantic hypotheses and instance space

For an attribute with k values, the syntax provides k specific values, one "?" and one "∅". Attendance has 3 values; each of the other five attributes has 2 values.

Number of syntactically distinct vectors = (3 + 2) × (2 + 2)^5 = 5 × 4^5 = 5,120.

Any vector containing "∅" in at least one position covers no instance. These syntactically different vectors therefore all represent the same empty concept. The vectors with no "∅" have one choice for "?" or a domain value at each attribute.

Number of non-empty semantic hypotheses = (3 + 1) × (2 + 1)^5 = 4 × 3^5 = 972.

Total semantically distinct hypotheses = 972 + 1 empty concept = 973.

Instance space size = 3 × 2 × 2 × 2 × 2 × 2 = 96.

The 973 semantic hypotheses form a restricted hypothesis language. They are not all possible Boolean concepts over the 96 instances; the unrestricted concept set would contain 2^96 possible labelings.

3.4 Generality ordering example

Hypothesis	Definition	Relation
h1	<Low, ?, ?, ?, ?, ?>	More general than h2.
h2	<Low, Irregular, Low, Weak, Passive, No>	More specific than h1 because it adds five constraints.
h3	<Medium, ?, ?, ?, ?, ?>	Incomparable with h1 and h2: it requires Medium attendance while both h1 and h2 require Low.

3.5 Consistency of the supplied hypotheses

Hypothesis	Positive examples covered	Negative examples covered	Consistent?
h_a = <Low, ?, ?, ?, Passive, ?>	E1, E2, E4	None	Yes. It covers all positive examples and no negative example.
h_b = <?, Irregular, ?, ?, ?, ?>	E1, E2, E4	E5	No. E5 is negative but is covered by h_b.
h_c = <Low, Irregular, Low, ?, ?, ?>	E1, E2	None	No. E4 is positive but is not covered because StudyHours = High.

A hypothesis satisfies an instance when it covers that individual instance. Consistency is a stronger, dataset-level property: every positive training example must be covered and every negative training example must be rejected. h_b is inconsistent because it covers the negative example E5. h_c is also inconsistent because it fails to cover the positive example E4. h_a is the only listed hypothesis consistent with all five examples.

TASK 4 – APPLY THE CANDIDATE ELIMINATION ALGORITHM (6 MARKS)

4.1 Initial boundaries

Initialise S0 with the maximally specific bottom hypothesis and G0 with the maximally general hypothesis:

S0 = {<∅, ∅, ∅, ∅, ∅, ∅>}; G0 = {<?, ?, ?, ?, ?, ?>}.

Notation used in the trace: L = Low, M = Medium, H = High, I = Irregular, R = Regular, Wk = Weak, St = Strong, P = Passive, A = Active, N = No, Y = Yes. A question mark means any domain value.

4.2 Step-by-step boundary trace

Step	Example / label	Action taken	S after update	G after update	Reason
1	E1 / Yes	Positive: set S from bottom to E1. G still covers E1.	{<L,I,L,Wk,P,N>}	{<?,?,?,?,?,?>}	No existing boundary is removed.
2	E2 / Yes	Generalise the PriorCGPA coordinate from Weak to ?.	{<L,I,L,?,P,N>}	{<?,?,?,?,?,?>}	The two positive examples differ only in PriorCGPA.
3	E3 / No	Specialise G minimally to exclude E3; retain only candidates at least as general as S.	{<L,I,L,?,P,N>}	{<L,?,?,?,?,?>; <?,I,?,?,?,?>; <?,?,L,?,?,?>; <?,?,?,?,P,?>; <?,?,?,?,?,N>}	Five minimal specialisations remain; details below.
4	E4 / Yes	Generalise StudyHours and Mentoring in S; delete G members not covering E4.	{<L,I,?,?,P,?>}	{<L,?,?,?,?,?>; <?,I,?,?,?,?>; <?,?,?,?,P,?>}	The StudyHours = Low and Mentoring = No specialisations are removed.

Step	Example / label	Action taken	S after update	G after update	Reason
5	E5 / No	Specialise the only G member that covers E5; remove dominated candidates.	{<L,I,?,?,P,?>}	{<L,?,?,?,?,?>; <?,?,?,?,P,?>}	New valid candidates duplicate the existing Low and Passive boundaries.

4.3 Generalisation on positive examples

E1 is the first positive example, so S1 becomes the exact E1 tuple. E2 shares Attendance = Low, Assignments = Irregular, StudyHours = Low, LMS = Passive and Mentoring = No with E1. Only PriorCGPA differs (Weak versus Strong), so the minimal generalisation replaces PriorCGPA with "?". E4 then differs from the S2 hypothesis in StudyHours (High rather than Low) and Mentoring (Yes rather than No). Both coordinates are minimally generalised to "?". The final specific boundary is S5 = {<Low, Irregular, ?, ?, Passive, ?>}.

4.4 Specialisation on the negative example E3

E3 is <High, Regular, High, Strong, Active, Yes>. G0 initially covers it, so each "?" coordinate can be specialised to a value other than the corresponding E3 value. A candidate is retained only if it remains at least as general as S2 = <Low, Irregular, Low, ?, Passive, No>.

Coordinate	Minimal alternative(s)	Decision
Attendance	Low, Medium	Keep Low. Reject Medium because it cannot cover S2, whose Attendance is Low.
Assignments	Irregular	Keep Irregular because it matches S2.
StudyHours	Low	Keep Low because it matches S2.
PriorCGPA	Weak	Reject Weak because S2 has "?" here; the specialisation would be narrower than S2.
LMS	Passive	Keep Passive because it matches S2.
Mentoring	No	Keep No because it matches S2.

Thus G3 contains five candidates: Low attendance, Irregular assignments, Low study hours, Passive LMS and No mentoring. No PriorCGPA specialisation survives. The version space after E3 contains only the S3 hypothesis because every surviving G member must also be more general than S3.

4.5 Positive filtering on E4

E4 is <Low, Irregular, High, Weak, Passive, Yes>. The G member that requires StudyHours = Low does not cover E4 and is removed. The G member that requires Mentoring = No also fails to cover E4 and is removed. The Low-attendance, Irregular-assignment and Passive-LMS candidates still cover E4 and remain in G4.

4.6 Specialisation on the negative example E5

E5 is <Medium, Irregular, High, Strong, Active, No>. The Low-attendance and Passive-LMS boundary hypotheses already reject E5, so they remain. The Irregular-Assignments hypothesis covers E5 and must be specialised. Attendance can be specialised to Low or High; Low is retained because it covers S4, while High is rejected because it is not at least as general as S4. StudyHours = Low, PriorCGPA = Weak and Mentoring = Yes are rejected because S4 has "?" at those positions. LMS = Passive is valid because it matches S4. These accepted specialisations duplicate the existing Low-attendance and Passive-LMS members; no new boundary member is needed.

4.7 Final boundaries

S5 = {<Low, Irregular, ?, ?, Passive, ?>}.

G5 = {<Low, ?, ?, ?, ?, ?>, <?, ?, ?, ?, Passive, ?>}.

TASK 5 – ANALYSE VERSION SPACE AND INDUCTIVE BIAS (4 MARKS)

5.1 Enumerate the final version space

A hypothesis belongs to the version space when it is at least as general as every member of S5 and no more general than every member of G5. Only two hypotheses satisfy both boundary constraints:

hG = <Low, ?, ?, ?, Passive, ?>

hS = <Low, Irregular, ?, ?, Passive, ?>

The version space size is 2. The general-to-specific lattice is shown below. hG is more general because it places no constraint on Assignments; hS adds Assignments = Irregular.

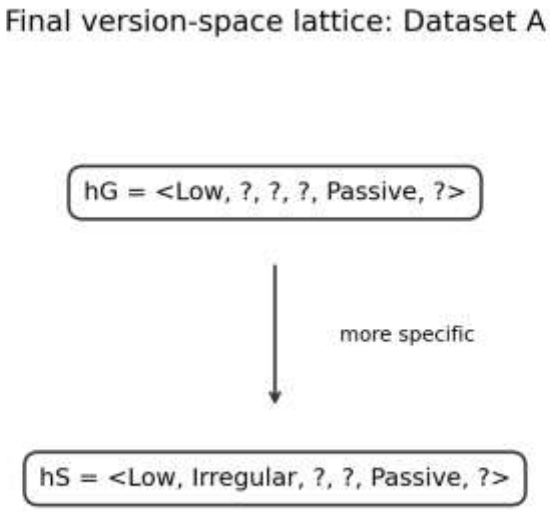


Figure 1. General-to-specific ordering of the two final version-space hypotheses.

The hypothesis <?, Irregular, ?, ?, ?, ?> is not in the version space. It covers E5, which is labelled No, so it is inconsistent with the training set. It also fails to enforce the Low attendance and Passive LMS conditions required by both final boundaries.

5.2 Query classification by version-space voting

Query	hG vote	hS vote	Vote total	Conclusion
Q1 = <Low, Regular, High, Weak, Passive, No>	Yes	No (Assignments is not Irregular)	1 Yes / 1 No	Ambiguous. The hypotheses disagree.

Query	hG vote	hS vote	Vote total	Conclusion
Q2 = <Low, Irregular, Low, Strong, Passive, Yes>	Yes	Yes	2 Yes / 0 No	Classified AtRisk = Yes by the entire version space.

Q1 is the informative query because its label would remove one of the two remaining hypotheses. Q2 is not informative for this version space because both hypotheses predict Yes. The most informative next labelling is any student with Attendance = Low and LMS = Passive but Assignments = Regular; Q1 is already such an example. Its label will decide whether the Irregular-assignment constraint is required.

5.3 Training data, hypothesis space, version space and inductive bias

Concept	Meaning in this assignment	Example
Training data	The observed labelled instances used to constrain learning.	E1–E5 in Dataset A.
Hypothesis space	All syntactically allowed candidate conjunctions (after merging semantically equivalent empty hypotheses, 973 concepts).	A candidate such as <Low, ?, ?, ?, Passive, ?>.
Version space	The subset of hypotheses consistent with every labelled training example.	The two hypotheses hG and hS after E1–E5.
Inductive bias	The restrictions and assumptions that make generalisation possible.	Only conjunctions of single-value constraints and "?" are representable; disjunctions are not.

5.4 Inductive bias demonstrated with the additional positive example

The assignment asks to add the positive instance <High, Regular, High, Strong, Passive, No> to E1–E5. This is a positive example with LMS = Passive, so the G member that requires Attendance = Low is removed. Generalising S to cover the new point removes the Low and Irregular constraints, while Passive remains common to every positive example.

S_new = G_new = {<?, ?, ?, ?, Passive, ?>}; |VS_new| = 1.

The version space shrinks from two hypotheses to one. The result demonstrates how the conjunctive bias narrows the possible concepts: it can retain a simple common condition (Passive LMS), but it cannot

express an arbitrary disjunction such as “Low attendance and Passive LMS, OR High attendance and Passive LMS” unless the representation is extended.

Unrestricted number of Boolean concepts over X_A = 2^96 = 79,228,162,514,264,337,593,543,950,336.

An unbiased learner over the full power set has no preference among these possible labelings. With only five examples, almost all unseen instances remain unconstrained. It therefore cannot generalise usefully without adding a bias such as conjunctions, trees, smoothness or a prior over concepts.

TASK 6 – CONSTRUCT THE DECISION TREE (6 MARKS)

6.1 Entropy of Dataset B

Dataset B contains 14 records: 7 AtRisk = Yes and 7 AtRisk = No. The class distribution is balanced.

$H(B) = -(7/14)\log_2(7/14) - (7/14)\log_2(7/14) = 1.0000$ bit.

6.2 Information gain for each root attribute

For an attribute A, the weighted entropy is the sum of each subset entropy multiplied by its fraction of the full set. Information gain is the parent entropy minus this weighted entropy.

$Gain(B, A) = H(B) - \Sigma_v (|B_v| / |B|) H(B_v)$.

Attribute	Value subsets: size (Yes, No), entropy	Weighted entropy	Information gain
Attendance	Low: 5 (4,1), 0.7219; Medium: 5 (2,3), 0.9710; High: 4 (1,3), 0.8113	0.8364	0.1636
Assignments	Irregular: 8 (5,3), 0.9544; Regular: 6 (2,4), 0.9183	0.9389	0.0611
LMS	Passive: 7 (5,2), 0.8631; Active: 7 (2,5), 0.8631	0.8631	0.1369
PriorCGPA	Weak: 7 (6,1), 0.5917; Strong: 7 (1,6), 0.5917	0.5917	0.4083

PriorCGPA has the highest information gain (0.4083 bits), so it is selected as the root. Its split divides the data into two equally sized but strongly class-skewed subsets: Weak contains 6 Yes and 1 No, while Strong contains 1 Yes and 6 No.

6.3 Recursive calculations for impure branches

Node / subset	Attribute candidate	Subset breakdown	Weighted entropy	Gain
PriorCGPA = Weak (7; 6Y,1N; H=0.5917)	Attendance	Low: 3 (3Y,0N), H=0; Medium: 2 (2Y,0N), H=0; High: 2 (1Y,1N), H=1	0.2857	0.3060
Same Weak subset	Assignments	Irregular: 4 (4Y,0N), H=0; Regular: 3 (2Y,1N), H=0.9183	0.3936	0.1981
Same Weak subset	LMS	Passive: 4 (4Y,0N), H=0; Active: 3 (2Y,1N), H=0.9183	0.3936	0.1981
PriorCGPA = Strong (7; 1Y,6N; H=0.5917)	Attendance	Low: 2 (1Y,1N), H=1; Medium: 3 (0Y,3N), H=0; High: 2 (0Y,2N), H=0	0.2857	0.3060

Node / subset	Attribute candidate	Subset breakdown	Weighted entropy	Gain
Same Strong subset	Assignments	Irregular: 4 (1Y,3N), H=0.8113; Regular: 3 (0Y,3N), H=0	0.4636	0.1281
Same Strong subset	LMS	Passive: 3 (1Y,2N), H=0.9183; Active: 4 (0Y,4N), H=0	0.3936	0.1981

For PriorCGPA = Weak, Attendance has the largest gain (0.3060). The Low and Medium branches are pure Yes leaves. The High branch has one Yes and one No; Assignments and LMS each split it perfectly with gain 1.0000. The tie is resolved in favour of Assignments because it appears first in the attribute order.

For PriorCGPA = Strong, Attendance again has the largest gain (0.3060). The Medium and High branches are pure No leaves. The Low branch has one Yes and one No; Assignments and LMS again both have gain 1.0000, so Assignments is selected by the fixed tie-breaking rule.

6.4 Final ID3 tree

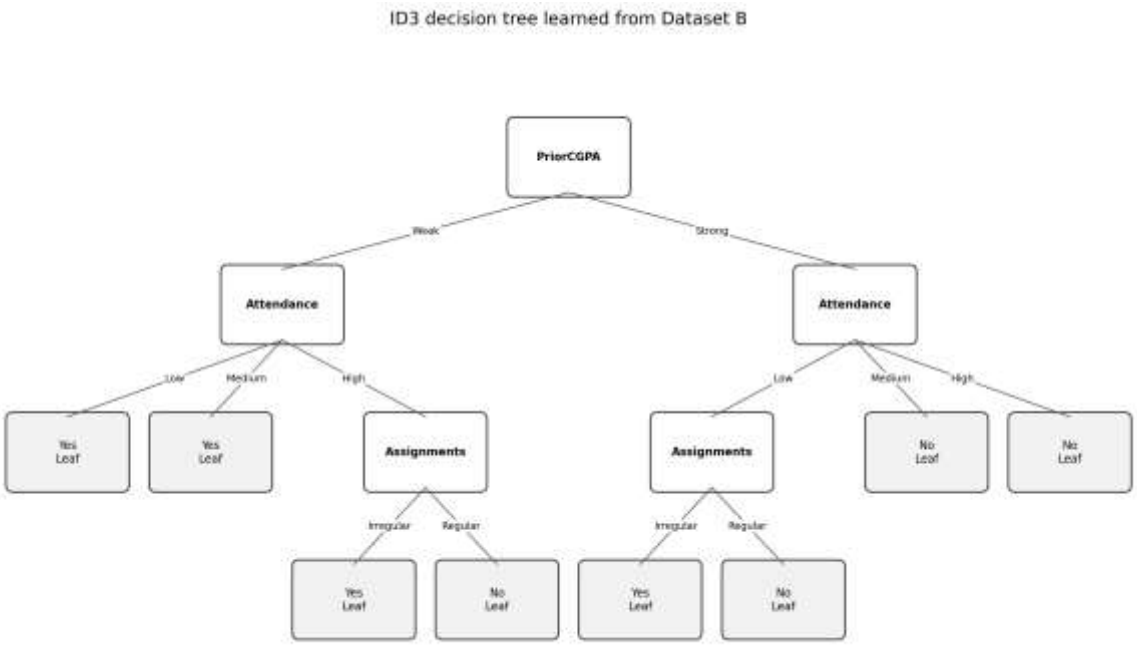


Figure 2. Dataset B decision tree constructed by the custom ID3 implementation.

6.5 IF–THEN rules

- IF PriorCGPA = Weak AND Attendance = Low THEN AtRisk = Yes.
- IF PriorCGPA = Weak AND Attendance = Medium THEN AtRisk = Yes.

- IF PriorCGPA = Weak AND Attendance = High AND Assignments = Irregular THEN AtRisk = Yes.
- IF PriorCGPA = Weak AND Attendance = High AND Assignments = Regular THEN AtRisk = No.
- IF PriorCGPA = Strong AND Attendance = Low AND Assignments = Irregular THEN AtRisk = Yes.
- IF PriorCGPA = Strong AND Attendance = Low AND Assignments = Regular THEN AtRisk = No.
- IF PriorCGPA = Strong AND Attendance = Medium THEN AtRisk = No.
- IF PriorCGPA = Strong AND Attendance = High THEN AtRisk = No.

Statistic	**Result**
Maximum depth	3
Decision / internal nodes	5
Leaf nodes	8
Total nodes	13
Training accuracy	14 / 14 = 100.0%

All training examples are correctly classified. This is a training result only; the held-out test evaluation in Task 8 shows that the fully grown tree does not generalise well to the six supplied test records.

TASK 7 – ANALYSE HEURISTIC SEARCH IN DECISION TREE LEARNING (3 MARKS)

7.1 ID3 as a search problem

Search element	ID3 interpretation in this assignment
Starting state	A single root leaf containing all 14 records in Dataset B.
Operator	Choose an unused attribute at a leaf, partition the records by its values and create child leaves.
Evaluation heuristic	Information gain in entropy; select the attribute with the largest gain.
Termination test	Stop when a subset is pure, no attributes remain, the depth limit is reached, the node has too few samples, or the best gain is below the threshold.

At the root ID3 evaluates 4 attributes. The Weak and Strong PriorCGPA branches each evaluate 3 remaining attributes. The two impure two-record branches (Weak/High and Strong/Low) each evaluate 2 remaining attributes. Therefore, 4 + 3 + 3 + 2 + 2 = 14 information-gain computations are performed. The ID3 construction visits only the local choices along its selected tree; it does not enumerate every possible tree.

Dataset B has 24 possible input combinations. There are 2^24 = 16,777,216 possible Boolean target concepts over those 24 points. This is a very large concept space compared with the 973 semantic conjunctive hypotheses counted in Task 3. The decision-tree hypothesis space also contains many possible tree structures for the same input attributes. ID3 controls this search through greedy selection and an inductive bias toward trees formed by high-gain local splits.

7.2 Information gain, gain ratio and StudentID

Root attribute	Information gain	Split information	Gain ratio
PriorCGPA	0.4083	1.0000	0.4083
Attendance	0.1636	1.5774	0.1037

PriorCGPA is much stronger than Attendance under information gain. Attendance has three values and divides the data into three groups, so its split information is higher. Gain ratio divides information gain by split information and penalises attributes that fragment the data into many subsets. This can reduce the preference for high-cardinality attributes, although gain ratio should still be used with safeguards.

StudentID: weighted child entropy = 0; IG = 1.0000 − 0 = 1.0000 bit.

SplitInfo(StudentID) = $\log_2(14)$ = 3.8074; GainRatio = 1.0000 / 3.8074 = 0.2626.

A unique StudentID creates fourteen one-record leaves and appears to classify the training set perfectly, but it memorises identities rather than academic-risk patterns. A new student has a new identifier and cannot be classified meaningfully by this split. This is a poor decision even though information gain is maximal. The adopted root criterion is information gain over the four meaningful attributes, with StudentID excluded from the predictor set by design.

7.3 Greedy tree versus an Attendance-rooted alternative

An alternative tree can be forced to use Attendance at the root and then use the same greedy ID3 rule inside each branch. The resulting tree uses Assignments under Attendance = Low, PriorCGPA under Attendance = Medium, and Assignments under Attendance = High. The impure two-record branches are split using the next available feature, with the deterministic attribute-order tie-break.

Tree	**Root**	**Total nodes**	**Leaves**	**Depth**	**Training accuracy**
Greedy ID3	PriorCGPA	13	8	3	100.0%
Alternative	Attendance	14	8	3	100.0%

The greedy choice produces a tree with one fewer node while keeping the same depth, leaf count and training accuracy. ID3 does not backtrack because it commits to the best local attribute, recurses on each subset and does not revisit earlier splits. This makes the search efficient, but a locally best split is not guaranteed to give the globally smallest tree or the best test accuracy.

Heuristic-search aspect	**Advantage or limitation**	**Dataset observation**
Advantage 1	Fast reduction of the search space.	ID3 evaluates 14 local gains rather than enumerating all possible trees.
Advantage 2	Produces readable rules.	The final rules use PriorCGPA, Attendance and Assignments and can be discussed with advisors.
Limitation 1	Greedy local choice may not be globally optimal.	Forcing Attendance at the root gives a tree with 14 rather than 13 nodes.
Limitation 2	High-cardinality or identifying attributes can overfit.	StudentID would obtain IG = 1 bit by isolating every training record.

TASK 8 – ANALYSE OVERFITTING AND UNDERFITTING (3 MARKS)

8.1 Training and test accuracy

Model	Complexity	Train accuracy	Test accuracy	Test confusion (TP,FP,TN,FN)	Diagnosis
M0	One majority leaf; depth 0	7/14 = 50.0%	3/6 = 50.0%	(3, 3, 0, 0)	Underfits: predicts Yes for every record due to the 7–7 tie.
M1	Maximum depth 1; 3 nodes, 2 leaves	12/14 = 85.7%	5/6 = 83.3%	(3, 1, 2, 0)	Best observed generalisation on this test set.
M2	Full tree; 13 nodes, 8 leaves, depth 3	14/14 = 100.0%	3/6 = 50.0%	(2, 2, 1, 1)	Overfits: perfect training fit but weak test performance.

M0 predicts the majority class at the root. Since the class counts are tied, the stated safety tie-break is Yes. M1 selects PriorCGPA at the root: Weak predicts Yes and Strong predicts No. M2 is the full tree from Task 6. M1 has the best observed test accuracy and avoids all false negatives on these six test records, although its sample is too small to establish a reliable 100% recall guarantee.

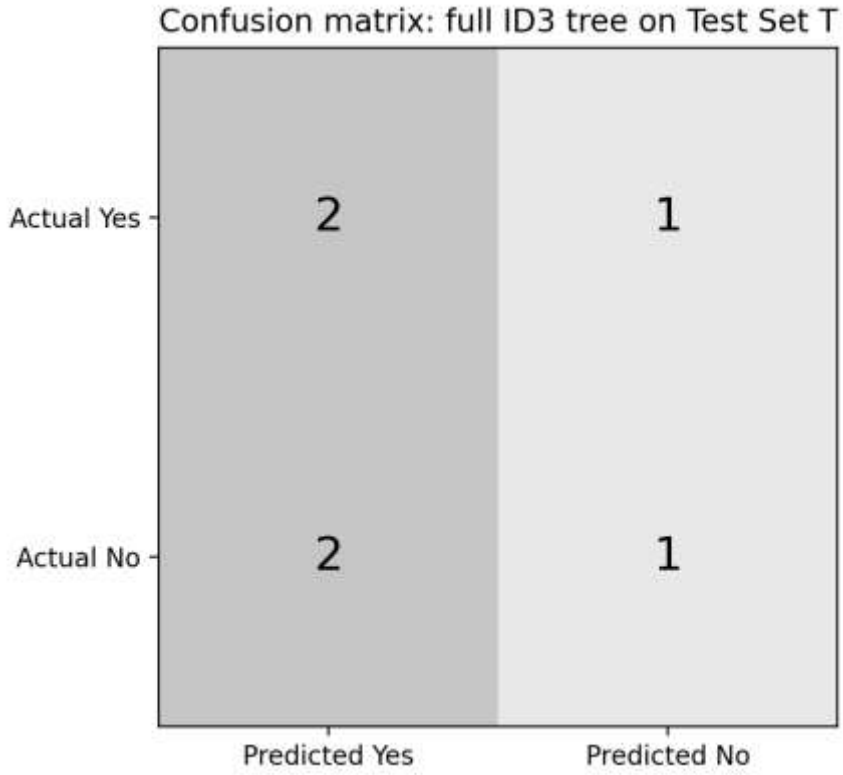


Figure 3. Confusion matrix of the fully grown ID3 model on Test Set T.

8.2 Causes of the observed behaviour

The training set contains only fourteen records and covers 14 of the 24 possible combinations. Several final leaves contain one or two examples, so a small number of records can determine a complete rule. A fully grown tree can memorise unusual combinations that do not repeat in Test Set T. Potential advisor-label inconsistency would increase this risk. The 100% training accuracy of M2 is therefore not enough to justify deployment.

8.3 Apply two complexity controls

Control 1: maximum depth = 1. This produces M1, a three-node tree with two leaves. Training accuracy is 85.7% and test accuracy is 83.3%. Its rule is PriorCGPA = Weak → Yes; PriorCGPA = Strong → No. The tie at the root is handled by the Yes-positive safety rule.

Control 2: minimum samples per split = 4. This stops the tree from splitting nodes containing fewer than four examples. The resulting tree has 9 nodes, 6 leaves and depth 2. Training accuracy is 85.7%; test accuracy is 50.0%. The small tied child leaves use Yes as the tie-break. This method reduces tree size but does not improve the test result on this particular six-record sample.

A validation-based pruning procedure would be preferable with a much larger validation set. Reduced-error pruning on only six records would be unstable, so the current assignment uses the transparent depth and minimum-split controls and reports their actual results.

8.4 Final complexity choice and evaluation limitation

For the supplied B/T experiment, M1 is the preferred baseline because it has the highest test accuracy (83.3%) and recall (100.0%) among M0–M2. M0 has 100% recall but only 50% precision and flags every student; M2 has lower recall (66.7%) and only 50% accuracy. The result should not be generalised to a real university because six test observations are too few: each single test error changes accuracy by 16.7 percentage points. The evaluation should be repeated over multiple cohorts using stratified cross-validation, a temporally later holdout and group-wise recall reporting.

TASK 9 – IMPLEMENT AND TEST THE ML TECHNIQUES (5 MARKS)

9.1 Implementation design

The companion file assignment1_reproducible.py contains the complete custom implementation. The code uses Python standard-library data structures and implements the learning logic directly. Matplotlib is used only for drawing experiment figures; no library routine is used for Candidate Elimination or the main ID3 tree.

Component	Implemented logic	Evidence
Hypothesis representation	Tuple of attribute constraints; "?" and "Ø" are supported.	covers() and more_general_or_equal() functions.
Candidate Elimination	S and G are maintained as sets. Positive examples minimally generalise S; negative examples minimally specialise G; dominated candidates are removed.	The trace is printed after each record and an empty version space is returned safely.
Decision tree	Entropy and information gain are calculated directly. ID3 recursively selects the best remaining attribute.	The implementation supports depth, min-samples-split and minimum-gain controls.
Prediction and rules	Prediction follows feature-value branches; unknown values use the parent-node majority class.	Rules are printed as IF–THEN paths.
Reproducibility	Fixed random seeds are used for Dataset C, split creation and label-noise injection.	The script writes a 500-row CSV and experiment plots.

9.2 Minimum test cases

Test	Input	Expected behaviour	Observed output	Pass / fail
TC1	Dataset A, E1→E5	S and G match the manual trace after each example.	Final S={<L,I,?,?,P,?>}; G={<L,?,?,?,?,?>; <?,?,?,?,P,?>}.	PASS

Test	Input	Expected behaviour	Observed output	Pass / fail
TC2	A pure subset; a 7-Yes / 7-No subset	Entropy is 0.0000 for pure data and 1.0000 for an even split.	0.0000 and 1.0000.	PASS
TC3	Dataset B, all four root attributes	Gains match Task 6 table.	Attendance .1636; Assignments .0611; LMS .1369; PriorCGPA .4083.	PASS
TC4	Build tree on Dataset B	All 14 training predictions match the labels and rules.	14 / 14 correct; 100.0%.	PASS
TC5	Predict Test Set T	Predictions match manual Task 8 calculation.	Yes, Yes, No, Yes, No, Yes; accuracy 50.0%.	PASS
TC6	Add a negative duplicate of E1	Candidate Elimination reports an empty version space without crashing.	G becomes empty; version-space length = 0.	PASS
TC7	Attendance = Very Low in a B-trained tree	Use the majority class at the parent node for an unseen branch.	Weak-PriorCGPA path falls back to Yes.	PASS
TC8	Weak/High branch where Assignments and LMS tie	Apply a deterministic feature-order tie-break.	Assignments selected because it appears before LMS.	PASS

9.3 Main intermediate and final outputs

Candidate Elimination prints S and G after every E1–E5 record. The final boundaries are the same as the hand trace in Task 4. The ID3 implementation prints the entropy/gain values for each candidate split, the selected attribute at each internal node, the final tree and the IF–THEN rules. The source script includes assertions for TC1–TC8; running it reports “TC1–TC8: PASS”.

The full reproducible Python source is included as Appendix B and is also supplied as a separate .py file so that it can be executed without copying code from the Word document. The generated 500-row Dataset C is supplied as a CSV file.

TASK 10 – EXPERIMENTAL ANALYSIS, THEORY VS EXPERIMENT, AND RECOMMENDATION (6 MARKS)

10A.1 Dataset characteristics

Dataset	Records	Attributes	Class balance (Yes / No)	Cardinalities	Unique-space coverage
A	5	6	3 / 2 (60% / 40%)	3, 2, 2, 2, 2, 2	5 / 96 = 5.2083%
B	14	4	7 / 7 (50% / 50%)	3, 2, 2, 2	14 / 24 = 58.3333%
T	6	4	3 / 3 (50% / 50%)	3, 2, 2, 2	6 / 24 = 25.0000%
C (synthetic)	500	4	251 / 249 (50.2% / 49.8%)	3, 2, 2, 2	24 / 24 = 100.0000%

Dataset C generator: each of the four categorical attributes is sampled uniformly from its stated domain. The label is generated from a documented risk score: +2 for Low attendance, +1 for Medium attendance, +1 for Irregular assignments, +1 for Passive LMS and +1 for Weak PriorCGPA. AtRisk is Yes when the total score is at least 3, otherwise No. A fixed random seed of 2026 produces 500 rows, 251 Yes and 249 No, and all 24 feature combinations occur. This synthetic data is designed for reproducible algorithm experiments; it is not a claim about real student outcomes.

10A.2 Held-out Test Set T and five-fold cross-validation

Evaluation	TP	FP	TN	FN	Accuracy	Precision	Recall	F1
Full ID3 on T (6 rows)	2	2	1	1	0.5000	0.5000	0.6667	0.5714
Stratified 5-fold on B + T (20 rows)	7	5	5	3	0.6000	0.5833	0.7000	0.6364

The five-fold evaluation uses B + T as the 20 labelled records, distributes each class across five folds in a deterministic round-robin order, trains a fresh custom ID3 tree on four folds and aggregates held-out predictions. The resulting confusion matrix is TP = 7, FP = 5, TN = 5 and FN = 3. Both evaluations show that the fully grown tree has poor recall for the small supplied dataset. The depth-1 result in Task 8 is better on T, which reinforces the need to control tree complexity.

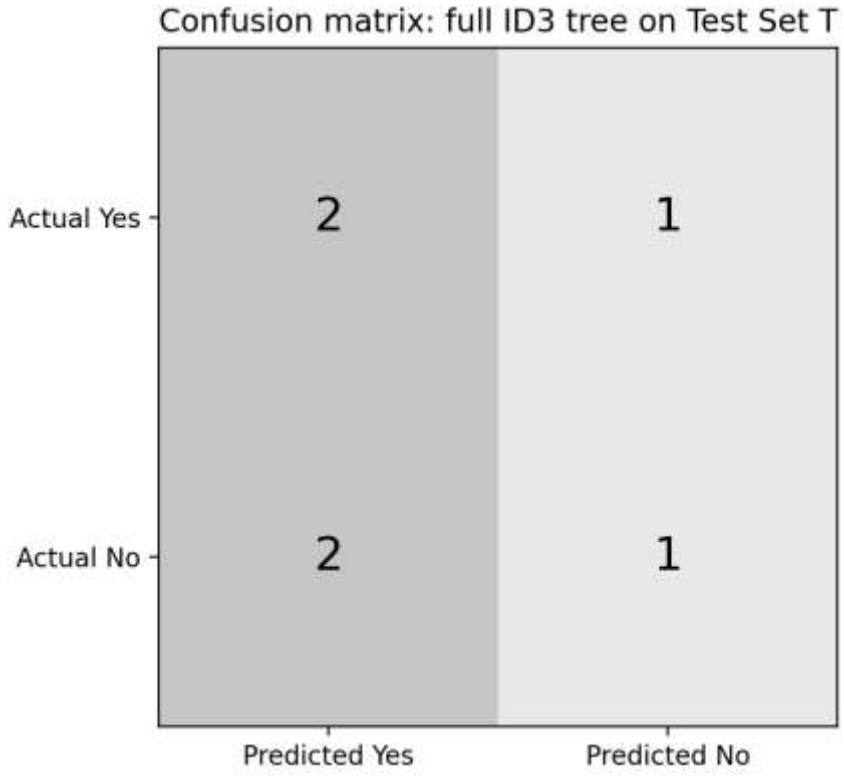


Figure 4. Full-tree confusion matrix on the six-record Test Set T.

10A.3 Learning curve

The learning curve is measured on Dataset C. For each training size, twenty deterministic subsamples are drawn from the 350-record training pool. Training accuracy and validation accuracy are averaged across the twenty runs; the same 150-record validation set is used for every sample size.

Training size	Mean training accuracy	Mean validation accuracy
4	1.0000	0.6873
6	1.0000	0.7463
8	1.0000	0.7597
10	1.0000	0.7713
12	1.0000	0.7723
14	1.0000	0.7907
25	1.0000	0.8630
50	1.0000	0.9580
100	1.0000	0.9950
200	1.0000	1.0000
350	1.0000	1.0000

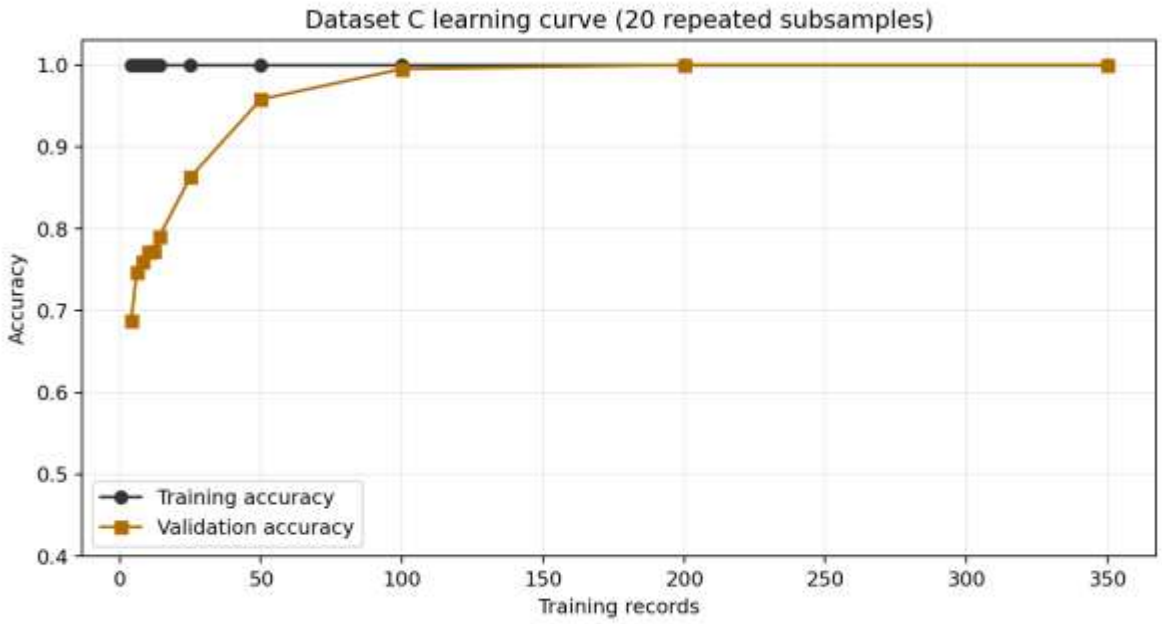


Figure 5. Dataset C learning curve. The training score remains perfect because the fully grown tree can isolate the observed training combinations; validation improves as more combinations are observed.

The increasing validation score is consistent with the fact that the synthetic dataset has only 24 possible feature combinations and the generator uses a fixed deterministic rule. With 200 training rows, all combinations are represented sufficiently for this particular rule. Real student data may contain more attributes, continuous values, missing values, changing relationships and label noise; the curve should not be interpreted as a deployment forecast.

10A.4 Effect of model complexity

Maximum depth	Nodes	Leaves	Training accuracy	Validation accuracy
0	1	1	0.5086	0.4733
1	4	3	0.7486	0.7533
2	10	6	0.8400	0.8400
3	18	10	0.8657	0.8667
4 (full)	26	14	1.0000	1.0000

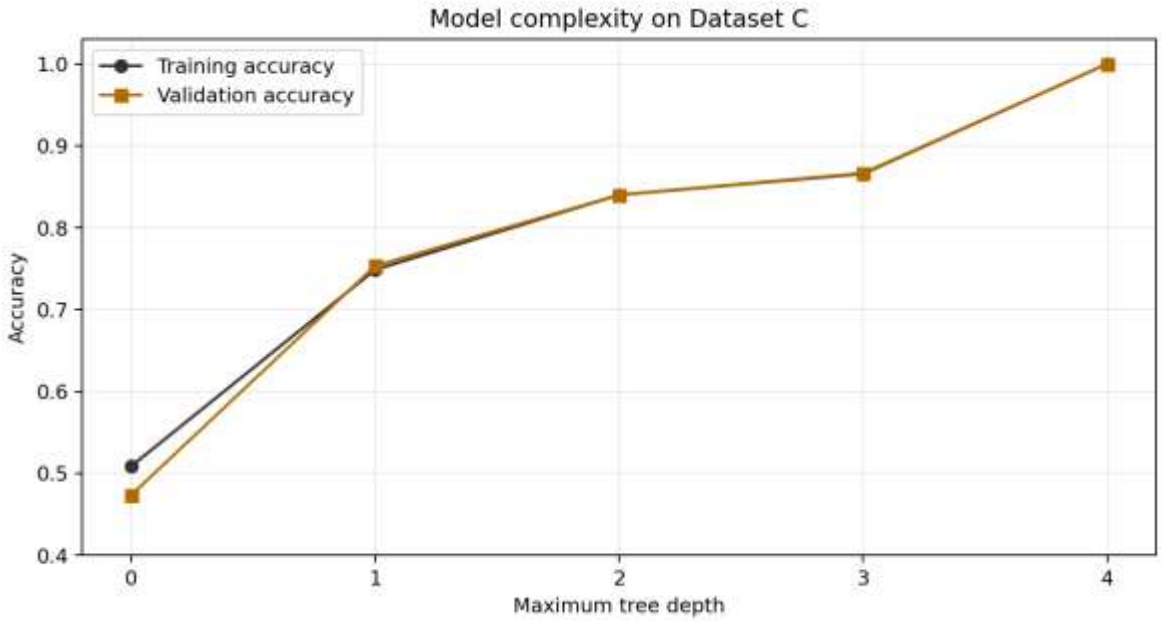


Figure 6. Accuracy against maximum depth on Dataset C.

On this synthetic problem, validation performance increases with depth because the full four-level tree can express the generator rule. This differs from the tiny B/T experiment where the full tree overfits. Model complexity cannot be judged independently of dataset size, noise, instance-space structure and validation design.

10A.5 Effect of label noise

Noise is injected by flipping exactly 0%, 5% or 10% of the 500 labels. The "fully grown" model has no effective depth/minimum-split restriction. The controlled model uses maximum depth 2, min_samples_split = 10 and minimum gain = 0.01. Validation accuracy in the table is measured against the correspondingly noised validation labels, while clean validation accuracy is also shown to make the synthetic ground-truth comparison explicit.

Noise	**Model**	**Nodes / leaves / depth**	**Training accuracy vs noisy labels**	**Validation accuracy vs noisy labels**	**Validation accuracy vs clean rule**
0%	Fully grown	26 / 14 / 4	1.0000	1.0000	1.0000
0%	Controlled	10 / 6 / 2	0.8400	0.8400	0.8400
5%	Fully grown	44 / 23 / 4	0.9514	0.9467	1.0000
5%	Controlled	10 / 6 / 2	0.7943	0.7600	0.8000

Noise	Model	Nodes / leaves / depth	Training accuracy vs noisy labels	Validation accuracy vs noisy labels	Validation accuracy vs clean rule
10%	Fully grown	46 / 24 / 4	0.8914	0.9200	1.0000
10%	Controlled	10 / 6 / 2	0.7657	0.7733	0.8400

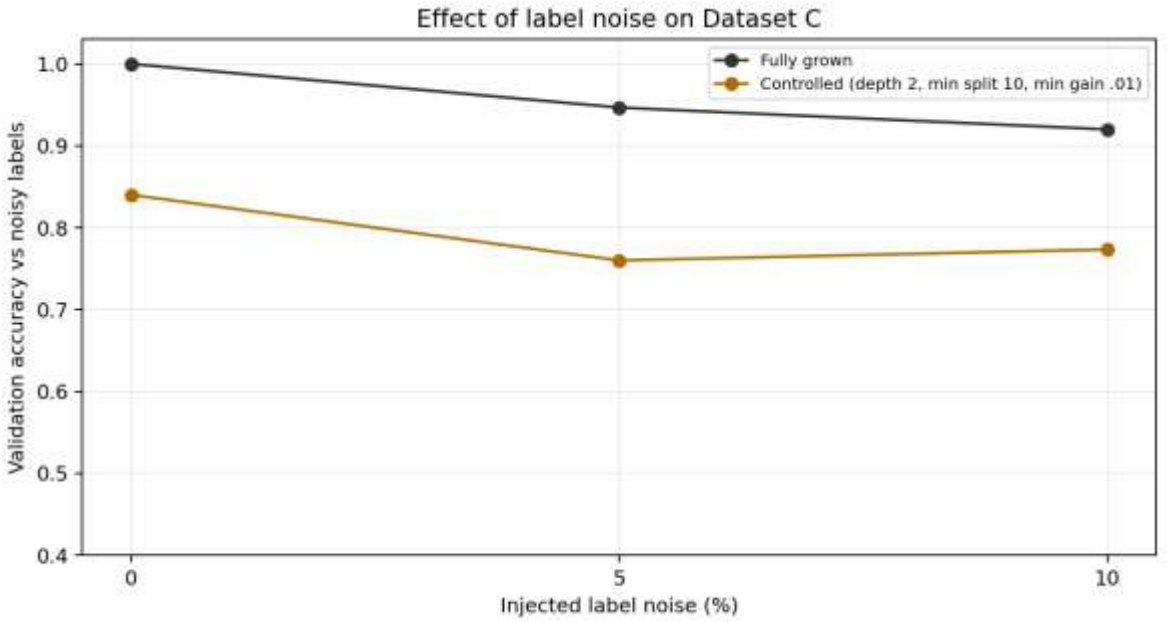


Figure 7. Validation accuracy against noisy validation labels at each injected noise rate.

The fully grown model becomes larger as the labels become noisier, growing from 26 to 44 and then 46 nodes. The controlled model stays at 10 nodes but has lower accuracy because its depth limit introduces bias. In this synthetic experiment, the fully grown model still obtains 100% clean-rule validation accuracy because all 24 possible combinations appear many times and the underlying rule is simple. This favourable outcome is specific to the generator; it does not prove that a fully grown tree is robust to real advisor noise.

10A.6 Candidate Elimination versus ID3 on Dataset B

When Candidate Elimination is run on Dataset B with the same four attributes, the version space collapses after example 4. The first three examples are positive and can be covered by the conjunction Attendance = Low. Example 4 is negative but also has Attendance = Low. The earlier positive records require the more specific hypothesis to remain general over Assignments, LMS and PriorCGPA; there is no conjunctive

specialisation that excludes example 4 while still covering all three positive examples. G becomes empty and no consistent conjunctive hypothesis remains.

Method	What it can represent	How it responds to noise / contradictions	Observed result
Candidate Elimination	A conjunction of one value or "?" per attribute; one constrained positive region.	Assumes a single deterministic concept. Conflicting labels can make the version space empty.	On Dataset B it collapses at example 4. On Dataset A it retains two hypotheses.
ID3 decision tree	A set of root-to-leaf conjunctions whose leaves form a disjunction of regions.	Can represent interactions and exceptions. Majority leaves and pruning can reduce sensitivity to isolated labels, although an unrestricted tree can memorise noise.	Fits all 14 B records; full tree test accuracy is 50.0%, while depth-1 accuracy is 83.3%.

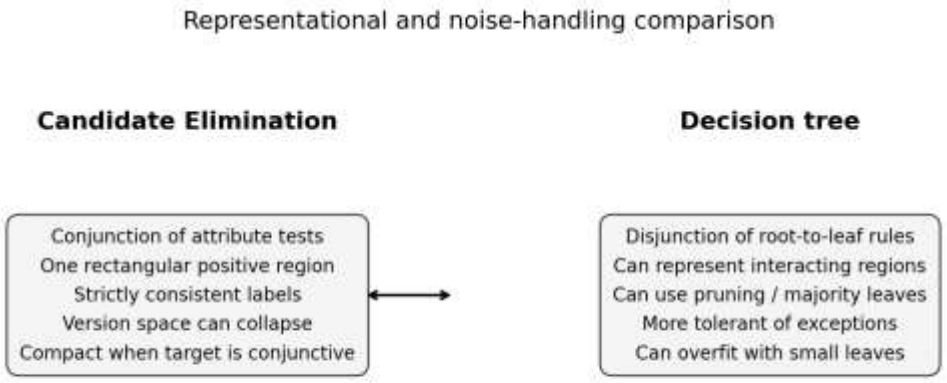


Figure 8. Representational and noise-handling comparison.

10B. Theory versus experimental behaviour

The theoretical prediction that a fully grown tree can overfit a small dataset is supported by Dataset B: training accuracy is 100.0%, but test accuracy falls to 50.0%. The depth-1 tree improves test accuracy to

83.3%. The six-record test is too small to establish a stable ranking, but it demonstrates why training fit must not be used alone.

Candidate Elimination behaves as expected under its conjunctive inductive bias. Dataset A has two consistent hypotheses, but Dataset B becomes inconsistent at example 4 because no single conjunction can include the positive examples and exclude the negative one. A decision tree can represent a disjunction of multiple conjunctions and therefore has a richer hypothesis language. That flexibility is useful for complex academic-risk patterns but increases overfitting risk.

The information-gain calculation selects PriorCGPA at the root. Gain ratio reduces the relative value of Attendance because Attendance has three branches and a larger split-information value. The StudentID example shows why an information-gain heuristic needs sensible feature screening: a unique identifier can create a perfect but non-generalising split.

The Dataset C experiments show that a larger sample improves validation accuracy under the known deterministic generator. They also show that noisy labels increase the size of a fully grown tree. The controlled tree is smaller but can underfit. These outcomes are consistent with the bias–variance trade-off, although the synthetic generator is much simpler than a university dataset.

Threats to validity include the very small real datasets, only six supplied test examples, the use of categorical features, the absence of missing values, the synthetic and deterministic Dataset C label rule, repeated combinations in C, and the possibility that real labels are affected by advisor policy. The validation split also comes from the same synthetic generator and should not be confused with an independent semester cohort.

10C. Evidence-based recommendation

For the original small-data version of the problem, a controlled decision tree is more suitable than Candidate Elimination. It can represent multiple disjoint rule paths, provides a human-readable path to each leaf and can be controlled through depth and minimum-sample settings. In the supplied B/T test, the depth-1 tree has accuracy 83.3%, precision 75.0%, recall 100.0% and F1 = 85.7%; the full tree has accuracy 50.0%, precision 50.0%, recall 66.7% and F1 = 57.1%. These measured results favour the depth-1 baseline for this particular split.

Candidate Elimination would be preferable only when the target is plausibly a single conjunction, the labels are highly reliable, the attribute domains are stable and the university wants an explicit version space of simple consistent rules. Those conditions do not hold after the Day-4 changes. The final deployment recommendation is therefore a modified, noise-tolerant tree-based model with class-weighted training, recall-based threshold selection, explicit unseen-value handling and human-readable

explanations. The 90% recall requirement must be verified on a sufficiently large, audited, temporally later validation cohort before release; the existing six-record test is not sufficient proof.

TASK 11 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT THE DESIGN (8 MARKS)

11A. Impact analysis

Changed condition	Original assumption violated	Impact category	Consequence
About 2,000 labelled records and weekly updates	Original training data is small and static.	Data and algorithm	More examples improve learning, but the pipeline must support update, monitoring and temporal validation.
5–10% inconsistent advisor labels	Labels are effectively noise-free and deterministic.	Data and algorithm	Strict Candidate Elimination can collapse; trees need noise controls and label auditing.
New value Attendance = Very Low	Attribute domains are fixed and fully known.	Data and hypothesis space	A model must support new categories or route them to a safe fallback.
Unseen attribute combinations	Training coverage is adequate for all future cases.	Hypothesis space and evaluation	The model must generalise and expose low-support cases for review.
At least 90% recall and an explanation for every flag	Accuracy alone is adequate.	Evaluation and deployment	Thresholds, cost-sensitive learning and per-student explanations become mandatory.

N2 and E6 test different failure modes. E6 is exactly the same input as E1 but has the opposite label. A deterministic concept learner cannot satisfy both labels, so Candidate Elimination removes every hypothesis and the version space is empty. A noise-tolerant tree can treat one of the duplicate labels as a noisy or disputed observation and use the majority or an audited label. N2 is a new combination (High, Regular, Active, Strong) labelled Yes; it does not duplicate an identical old feature vector with an opposite label. It may indicate a new interaction or an exception, so a tree can add a branch or leaf for it. A noise-tolerant tree should still check whether N2 is a genuine new pattern or a mistaken label before expanding the model.

11B. Re-evaluation of the original design

11B.1 Candidate Elimination with E6

Starting from E1–E5, the final boundaries are S5 = {<Low, Irregular, ?, ?, Passive, ?>} and G5 = {<Low, ?, ?, ?, ?, ?>; <?, ?, ?, ?, Passive, ?>}. E6 has the same values as E1 but AtRisk = No. Every specialisation that would exclude E6 also excludes E1, which is a positive example. No valid member can remain in G.

After E6: S = {<Low, Irregular, ?, ?, Passive, ?>}; G = ∅; |VS| = 0.

The algorithm reports an empty version space. It does not crash or invent a new rule. This is the expected result under a strict noise-free conjunction assumption.

11B.2 Candidate Elimination with E7 then E8 (cumulative, after E1–E5)

Update	New positive example	S boundary	G boundary	Version-space size
After E7	<Medium, Regular, High, Weak, Passive, No>	{<?, ?, ?, ?, Passive, ?>}	{<?, ?, ?, ?, Passive, ?>}	1
After E8	<Low, Regular, High, Strong, Active, Yes>	{<?, ?, ?, ?, ?, ?>}	∅	0

E7 is a positive example with LMS = Passive and is compatible with the final generalisation shared by the earlier positive examples. It collapses the version space to the single Passive-LMS hypothesis. E8 is also positive but has LMS = Active, so the only remaining G member fails to cover it. The general boundary becomes empty and the version space collapses. This shows the limitation of one conjunction: it cannot retain a single LMS value while covering both positive groups.

11B.3 Rebuild the decision tree after N1–N3

The three new records are added to Dataset B. The expanded Attendance domain is Very Low, Low, Medium and High. The ID3 tree is rebuilt using the same entropy calculation and deterministic feature-order tie-break. The root remains PriorCGPA. The new tree has 19 nodes, 12 leaves and maximum depth 4, compared with the original 13 nodes, 8 leaves and depth 3. It classifies all 17 training records correctly, but this is training accuracy only.

New record	Input and label	Original B-trained tree	Updated B+N tree
N1	Very Low, Irregular, Passive, Weak → Yes	Predicts Yes by fallback to the majority class at the Weak node.	Creates an explicit Very Low → Yes branch.
N2	High, Regular, Active, Strong → Yes	Predicts No under Strong / High.	Creates a deeper Strong / High / Regular / LMS split; Active → Yes.
N3	Low, Regular, Passive, Weak → Yes	Predicts Yes under Weak / Low.	Remains Yes in the Weak / Low branch.

The original tree handles N1 as an unseen Attendance value: at the PriorCGPA = Weak node, the unseen branch falls back to its local majority class Yes. After training includes N1, the tree has an explicit Very Low branch. N2 changes an existing Strong / High / Regular region that previously ended as No; the rebuilt model adds an LMS test to separate the new positive example (Active) from the earlier negative example (Passive).

The original approach cannot be said to satisfy the new 90% recall requirement from this experiment. The original full tree has 66.7% recall on Test Set T, while the depth-1 tree has 100% recall on only six test examples. The updated tree reaches 100% training recall on the 17 records but has no independent post-update holdout. Candidate Elimination fails on E6 and E8, so it is unsuitable for the changed label conditions. Decision trees can provide path explanations, but the recall target still requires independent validation and threshold tuning.

11C. Adaptation / redesign

Decision: REDESIGN the production pipeline while retaining a decision-tree baseline for transparency. Candidate Elimination is retained only as a teaching and diagnostic tool for small, clean, deterministic datasets. It is removed from the production classification path because the new problem includes noisy labels, new values and disjunctive patterns.

11C.1 Revised model and data pipeline

- Validate the input schema and preserve measurement time. Convert attendance percentages into the defined ordinal bins, add an explicit Unknown / Other category and log previously unseen values such as Very Low.

- Audit duplicate feature vectors with conflicting labels. Send a sample of advisor disagreements for adjudication; retain label provenance and avoid silently treating every label as equally reliable.
- Train a cost-sensitive tree ensemble (for example, a class-weighted Random Forest) as the risk scorer, and keep a pruned decision tree as an interpretable reference model. Use temporal validation and group-wise evaluation.
- Choose the decision threshold on a validation cohort to achieve recall of at least 0.90 for AtRisk = Yes. A lower threshold or higher positive-class weight will usually increase recall but can reduce precision and increase the number of students referred for review.
- For every flagged student, generate a human-readable explanation: the risk score, top contributing inputs, the applicable decision path or rule, and a note that the output is a support prompt rather than a final academic decision. SHAP-style feature contributions can supplement a short rule-based explanation.
- If a student has an unseen value or a low-support combination, route the record to manual review rather than automatically predicting No. Add audited new cases to the next training cycle.

11C.2 High-level algorithm outline

INPUT: Week-6 student record x; audited historical set D; target recall r = 0.90.

1. Validate field names, timestamps and value domains. Map unknown categorical values to Unknown and raise a review flag.

2. Check D for duplicate feature vectors with conflicting labels; retain provenance and adjudicated labels where available.

3. Split D by semester into training, validation and later-time test cohorts.

4. Train a class-weighted tree ensemble and a pruned decision-tree baseline.

5. On validation data, select the lowest probability threshold τ for which AtRisk recall is at least r; record the accompanying precision.

6. For x, calculate risk probability p(x). If a domain value is unseen or the leaf support is too small, request human review.

7. If $p(x) \geq \tau$, flag for support and produce a rule path plus top feature contributions; otherwise retain the record for monitoring.

8. Monitor recall, precision, calibration, group-wise error rates, label disagreement and unseen-value frequency. Retrain only after drift and label-quality checks.

OUTPUT: risk estimate, flag / review status and human-readable explanation.

11C.3 Expected effect and trade-off

The revised system can represent disjunctions and interactions through multiple tree paths. Ensemble averaging and class weighting are more tolerant of isolated inconsistent labels than strict Candidate Elimination, but the ensemble is less directly interpretable than a single tree. Threshold tuning is designed to meet the 90% recall constraint; it will normally increase false positives and reduce precision. This is an acceptable trade-off only if advisors have enough capacity and explanations are presented as support evidence, not as proof that a student will fail.

11D. Evidence-based defence and monitoring

The revised design follows from the evidence. Candidate Elimination has a strong conjunctive bias and a very small version space on Dataset A, but it collapses as soon as contradictory or disjunctive examples are added. ID3 represents unions of path conditions and can fit the supplied small data, but its full tree overfits: 100% training accuracy and 50% test accuracy. A depth-1 tree improves the observed test metrics, while Dataset C shows that increasing depth can be useful when the underlying generator is deterministic and sufficiently represented. The noise experiment shows that a fully grown tree becomes more complex when labels are flipped, whereas a controlled tree remains smaller but can underfit.

The Day-4 records also show why a fixed-domain, noise-free system is not adequate. The updated decision tree grows from 13 to 19 nodes and reaches 100% accuracy on the 17 training records, but no post-update independent evaluation is supplied. Therefore, neither the 90% recall requirement nor robust performance on new attendance values has been proven. The redesigned model must be accepted only after a temporally separated validation cohort demonstrates recall ≥ 0.90 and the associated precision remains operationally manageable.

Remaining limitations include advisor-label disagreement, possible bias in attendance and LMS variables, distribution drift, and explanations that may be mistaken for causal reasons. After deployment, monitor (i) recall on an audited sample of completed semesters, (ii) precision and advisor review workload, (iii) label disagreement, (iv) new-value and low-support rates, (v) calibration and risk-score drift, and (vi) error rates across student groups. Trigger a review and retraining cycle when recall falls below 0.90 or when a material shift is detected. A human advisor must retain responsibility for all support decisions.

APPENDIX A – CORE EQUATIONS AND NOTATION

Entropy

$H(S) = -\sum_i p_i \log_2(p_i)$, with $0 \log_2(0)$ defined as 0.

Information gain

$IG(S, A) = H(S) - \sum_v (|S_v| / |S|) H(S_v)$.

Split information and gain ratio

$SplitInfo(S, A) = -\sum_v (|S_v| / |S|) \log_2(|S_v| / |S|)$.

$GainRatio(S, A) = IG(S, A) / SplitInfo(S, A)$.

Classification metrics

$Accuracy = (TP + TN) / (TP + FP + TN + FN)$.

$Precision = TP / (TP + FP)$; $Recall = TP / (TP + FN)$.

$F1 = 2 \times Precision \times Recall / (Precision + Recall)$.

TP = true positive; FP = false positive; TN = true negative; FN = false negative. AtRisk = Yes is the positive class throughout this report.

APPENDIX B – COMPLETE REPRODUCIBLE PYTHON IMPLEMENTATION

The complete runnable implementation is also supplied in the companion file assignment1_reproducible.py. Run it with Python 3 using: python assignment1_reproducible.py. Matplotlib is required only to save the experiment plots; the custom learning algorithms use the Python standard library. The script creates dataset_C_synthetic.csv and the learning-curve, complexity and noise figures.

The source listing below is included for editability. It is the same implementation supplied as the separate .py file.

```
"""ITA06 Machine Learning - Assignment 1 reproducibility script
Implements Candidate Elimination and an ID3-style decision tree from first
principles. No library Candidate Elimination or decision-tree learner is used.
Run:
    python assignment1_reproducible.py
Outputs:
    - console traces and test results
    - dataset_C_synthetic.csv (500 rows)
    - figures/learning_curve.png, figures/tree_complexity.png,
      figures/noise_effect.png and figures/test_confusion_matrix.png
Reproducibility: Python standard-library random generator; fixed seeds are
specified at each data-generation/sampling step.
"""
from __future__ import annotations
import csv
import itertools
import math
import random
from collections import Counter, defaultdict
from dataclasses import dataclass, field
from pathlib import Path
from typing import Dict, Iterable, List, Optional, Sequence, Tuple
ROOT = Path(__file__).resolve().parent
FIG_DIR = ROOT / "figures"
FIG_DIR.mkdir(exist_ok=True)
```

```
# ---------------------------------------------------------------------
# Source datasets transcribed from the assignment sheet
# ---------------------------------------------------------------------
A_ATTRIBUTES = ["Attendance", "Assignments", "StudyHours", "PriorCGPA", "LMS", "Mentoring"]
A_DOMAINS = [
    ["Low", "Medium", "High"], ["Irregular", "Regular"], ["Low", "High"],
    ["Weak", "Strong"], ["Passive", "Active"], ["No", "Yes"],
]
DATASET_A = [
    ("E1", ("Low", "Irregular", "Low", "Weak", "Passive", "No"), "Yes"),
    ("E2", ("Low", "Irregular", "Low", "Strong", "Passive", "No"), "Yes"),
    ("E3", ("High", "Regular", "High", "Strong", "Active", "Yes"), "No"),
    ("E4", ("Low", "Irregular", "High", "Weak", "Passive", "Yes"), "Yes"),
    ("E5", ("Medium", "Irregular", "High", "Strong", "Active", "No"), "No"),
]
B_ATTRIBUTES = ["Attendance", "Assignments", "LMS", "PriorCGPA"]
B_DOMAINS = [
    ["Low", "Medium", "High"], ["Irregular", "Regular"],
    ["Passive", "Active"], ["Weak", "Strong"],
]
DATASET_B = [
    ("1", ("Low", "Irregular", "Passive", "Weak"), "Yes"),
    ("2", ("Low", "Irregular", "Passive", "Strong"), "Yes"),
    ("3", ("Low", "Regular", "Active", "Weak"), "Yes"),
    ("4", ("Low", "Regular", "Active", "Strong"), "No"),
    ("5", ("Medium", "Irregular", "Passive", "Weak"), "Yes"),
    ("6", ("Medium", "Irregular", "Active", "Strong"), "No"),
    ("7", ("Medium", "Regular", "Passive", "Weak"), "Yes"),
    ("8", ("Medium", "Regular", "Active", "Strong"), "No"),
    ("9", ("High", "Irregular", "Passive", "Weak"), "Yes"),
    ("10", ("High", "Irregular", "Active", "Strong"), "No"),
    ("11", ("High", "Regular", "Passive", "Strong"), "No"),
    ("12", ("High", "Regular", "Active", "Weak"), "No"),
```

```
    ("13", ("Medium", "Irregular", "Passive", "Strong"), "No"),
    ("14", ("Low", "Irregular", "Active", "Weak"), "Yes"),
]
TEST_SET_T = [
    ("T1", ("Low", "Irregular", "Active", "Strong"), "No"),
    ("T2", ("Medium", "Regular", "Active", "Weak"), "No"),
    ("T3", ("High", "Regular", "Passive", "Weak"), "Yes"),
    ("T4", ("Medium", "Irregular", "Active", "Weak"), "Yes"),
    ("T5", ("Low", "Regular", "Passive", "Strong"), "No"),
    ("T6", ("High", "Irregular", "Active", "Weak"), "Yes"),
]
# ------------------------------------------------------------------------
# Candidate Elimination
# ------------------------------------------------------------------------
Hypothesis = Tuple[str, ...]
def covers(h: Hypothesis, x: Sequence[str]) -> bool:
    """Return True when conjunctive hypothesis h covers instance x."""
    return all(value == "?" or value == x[i] for i, value in enumerate(h))
def more_general_or_equal(h1: Hypothesis, h2: Hypothesis) -> bool:
    """Whether h1 is at least as general as h2 (including bottom symbols)."""
    for a, b in zip(h1, h2):
        if b == "∅":
            continue
        if a == "?":
            continue
        if a != b:
            return False
    return True
def minimal_generalization(s: Hypothesis, x: Sequence[str]) -> Hypothesis:
    result = []
    for i, value in enumerate(s):
        if value == "∅":
            result.append(x[i])
```

```
        elif value == x[i] or value == "?":
            result.append(value)
        else:
            result.append("?")
    return tuple(result)
def minimal_specializations(g: Hypothesis, x: Sequence[str], domains: Sequence[Sequence[str]], s_set:
set) -> set:
    result = set()
    for i, value in enumerate(g):
        if value != "?":
            continue
        for alternative in domains[i]:
            if alternative == x[i]:
                continue
            candidate = list(g)
            candidate[i] = alternative
            candidate = tuple(candidate)
            # A specialization must remain at least as general as every S boundary member.
            if all(more_general_or_equal(candidate, s) for s in s_set):
                result.add(candidate)
    return result
def enumerate_version_space(s_set: set, g_set: set, domains: Sequence[Sequence[str]]) ->
List[Hypothesis]:
    if not s_set or not g_set:
        return []
    choices = [list(domain) + ["?"] for domain in domains]
    candidates = itertools.product(*choices)
    return [h for h in candidates
            if all(more_general_or_equal(g, h) for g in g_set)
            and all(more_general_or_equal(h, s) for s in s_set)]
def candidate_elimination(data, domains):
    n = len(domains)
    S = {tuple("∅" for _ in range(n))}
```

```
    G = {tuple("?" for _ in range(n))}
    trace = []
    for name, x, label in data:
        if label == "Yes":
            G = {g for g in G if covers(g, x)}
            new_s = {minimal_generalization(s, x) if not covers(s, x) else s for s in S}
            S = {s for s in new_s if not any(s != other and more_general_or_equal(s, other)
                                             for other in new_s)}
        else:
            new_g = set()
            for g in G:
                if not covers(g, x):
                    new_g.add(g)
                else:
                    new_g |= minimal_specializations(g, x, domains, S)
            G = {g for g in new_g if not any(g != other and more_general_or_equal(other, g)
                                             for other in new_g)}
        trace.append({"example": name, "label": label, "S": sorted(S), "G": sorted(G),
                      "version_space": enumerate_version_space(S, G, domains)})
    return S, G, trace
# ---------------------------------------------------------------------------
# ID3 decision tree implemented from entropy and information gain
# ---------------------------------------------------------------------------
@dataclass
class TreeNode:
    attribute: Optional[int] = None
    prediction: Optional[str] = None
    branches: Dict[str, "TreeNode"] = field(default_factory=dict)
    counts: Counter = field(default_factory=Counter)
    default_prediction: str = "Yes"
    depth: int = 0
def entropy(rows) -> float:
    counts = Counter(label for _, _, label in rows)
```

```
    n = len(rows)
    if n == 0:
        return 0.0
    return -sum((count / n) * math.log2(count / n) for count in counts.values() if count)
def majority_class(rows, default="Yes") -> str:
    counts = Counter(label for _, _, label in rows)
    if not counts:
        return default
    # Conservative tie-break for this application: predict Yes to avoid a false negative.
    return "Yes" if counts["Yes"] >= counts["No"] else "No"
def split_by_attribute(rows, attribute_index):
    subsets = defaultdict(list)
    for row in rows:
        subsets[row[1][attribute_index]].append(row)
    weighted_entropy = sum(len(subset) / len(rows) * entropy(subset)
                           for subset in subsets.values())
    return subsets, entropy(rows) - weighted_entropy
def build_id3(rows, attributes, domains, features=None, max_depth=None,
              min_samples_split=2, min_gain=1e-12, depth=0, default="Yes"):
    if features is None:
        features = list(range(len(attributes)))
    counts = Counter(label for _, _, label in rows)
    majority = majority_class(rows, default)
    if len(counts) <= 1:
        pure_label = next(iter(counts)) if counts else default
        return TreeNode(prediction=pure_label, counts=counts,
                        default_prediction=majority, depth=depth)
    if (not features or (max_depth is not None and depth >= max_depth)
            or len(rows) < min_samples_split):
        return TreeNode(prediction=majority, counts=counts,
                        default_prediction=majority, depth=depth)
    gains = [(split_by_attribute(rows, i)[1], i) for i in features]
    # Deterministic tie-break: first attribute in the supplied feature order.
```

```
    best_gain, best_attribute = max(gains, key=lambda item: (item[0], -features.index(item[1])))
    if best_gain < min_gain:
        return TreeNode(prediction=majority, counts=counts,
                        default_prediction=majority, depth=depth)
    subsets, _ = split_by_attribute(rows, best_attribute)
    node = TreeNode(attribute=best_attribute, counts=counts,
                    default_prediction=majority, depth=depth)
    remaining = [i for i in features if i != best_attribute]
    for value in domains[best_attribute]:
        if value in subsets:
            node.branches[value] = build_id3(
                subsets[value], attributes, domains, remaining, max_depth,
                min_samples_split, min_gain, depth + 1, majority)
        else:
            node.branches[value] = TreeNode(prediction=majority, counts=Counter(),
                                            default_prediction=majority, depth=depth + 1)
    return node
def predict_tree(node: TreeNode, instance: Sequence[str]) -> str:
    while node.attribute is not None:
        value = instance[node.attribute]
        # Unseen values use the majority class at the parent node; this is documented
        # and deliberately conservative for an early-warning workflow.
        if value not in node.branches:
            return node.default_prediction
        node = node.branches[value]
    return node.prediction or "Yes"
def tree_statistics(node: TreeNode):
    if node.attribute is None:
        return 1, 1, node.depth
    child_stats = [tree_statistics(child) for child in node.branches.values()]
    return (1 + sum(item[0] for item in child_stats),
            sum(item[1] for item in child_stats),
            max(item[2] for item in child_stats))
```

```
def print_rules(node: TreeNode, attributes: Sequence[str], conditions=None):
    conditions = conditions or []
    if node.attribute is None:
        left = " AND ".join(conditions) if conditions else "TRUE"
        print(f"IF {left} THEN AtRisk = {node.prediction}")
        return
    attribute = attributes[node.attribute]
    for value, child in node.branches.items():
        print_rules(child, attributes, conditions + [f"{attribute} = {value}"])
def classification_metrics(rows, model, attributes):
    tp = fp = tn = fn = 0
    predictions = []
    for _, instance, actual in rows:
        predicted = predict_tree(model, instance)
        predictions.append((actual, predicted))
        if actual == "Yes" and predicted == "Yes": tp += 1
        elif actual == "No" and predicted == "Yes": fp += 1
        elif actual == "No" and predicted == "No": tn += 1
        else: fn += 1
    precision = tp / (tp + fp) if tp + fp else 0.0
    recall = tp / (tp + fn) if tp + fn else 0.0
    f1 = 2 * precision * recall / (precision + recall) if precision + recall else 0.0
    return {"TP": tp, "FP": fp, "TN": tn, "FN": fn,
            "accuracy": (tp + tn) / len(rows), "precision": precision,
            "recall": recall, "f1": f1, "predictions": predictions}
# ---------------------------------------------------------------------------
# Synthetic scalability dataset C
# ---------------------------------------------------------------------------
def synthetic_risk_label(instance):
    attendance, assignments, lms, cgpa = instance
    risk_score = (2 if attendance == "Low" else (1 if attendance == "Medium" else 0))
    risk_score += 1 if assignments == "Irregular" else 0
    risk_score += 1 if lms == "Passive" else 0
```

```
    risk_score += 1 if cgpa == "Weak" else 0
    return "Yes" if risk_score >= 3 else "No"
def generate_dataset_c():
    random.seed(2026)
    rows = []
    for i in range(500):
        instance = tuple(random.choice(domain) for domain in B_DOMAINS)
        rows.append((f"C{i+1}", instance, synthetic_risk_label(instance)))
    return rows
def write_csv(rows, path):
    with open(path, "w", newline="", encoding="utf-8") as handle:
        writer = csv.writer(handle)
        writer.writerow(["StudentID"] + B_ATTRIBUTES + ["AtRisk"])
        for name, instance, label in rows:
            writer.writerow([name, *instance, label])
# ---------------------------------------------------------------------------
# Test suite
# ---------------------------------------------------------------------------
def cross_validate(rows, k=5):
    """Deterministic stratified k-fold evaluation; returns aggregate confusion metrics."""
    folds = [[] for _ in range(k)]
    by_class = defaultdict(list)
    for index, row in enumerate(rows):
        by_class[row[2]].append(index)
    for class_rows in by_class.values():
        for position, index in enumerate(class_rows):
            folds[position % k].append(index)
    actual_all, predicted_all = [], []
    for fold in folds:
        test_indices = set(fold)
        train = [row for i, row in enumerate(rows) if i not in test_indices]
        test = [row for i, row in enumerate(rows) if i in test_indices]
        model = build_id3(train, B_ATTRIBUTES, B_DOMAINS)
```

```
        for _, instance, actual in test:
            actual_all.append(actual)
            predicted_all.append(predict_tree(model, instance))
    tp = sum(a == "Yes" and p == "Yes" for a, p in zip(actual_all, predicted_all))
    fp = sum(a == "No" and p == "Yes" for a, p in zip(actual_all, predicted_all))
    tn = sum(a == "No" and p == "No" for a, p in zip(actual_all, predicted_all))
    fn = sum(a == "Yes" and p == "No" for a, p in zip(actual_all, predicted_all))
    precision = tp / (tp + fp) if tp + fp else 0.0
    recall = tp / (tp + fn) if tp + fn else 0.0
    f1 = 2 * precision * recall / (precision + recall) if precision + recall else 0.0
    return {"TP": tp, "FP": fp, "TN": tn, "FN": fn,
            "accuracy": (tp + tn) / len(rows), "precision": precision,
            "recall": recall, "f1": f1}
def run_tests():
    # TC1: Candidate Elimination trace agrees with the hand-worked boundary result.
    s, g, trace = candidate_elimination(DATASET_A, A_DOMAINS)
    assert trace[0]["S"] == [("Low", "Irregular", "Low", "Weak", "Passive", "No")]
    assert s == {("Low", "Irregular", "?", "?", "Passive", "?")}
    assert set(g) == {("Low", "?", "?", "?", "?", "?"),
                      ("?", "?", "?", "?", "Passive", "?")}
    # TC2: entropy of pure and evenly split subsets.
    assert abs(entropy([("a", (), "Yes") for _ in range(4)]) - 0.0) < 1e-12
    balanced = [(str(i), (), "Yes" if i < 7 else "No") for i in range(14)]
    assert abs(entropy(balanced) - 1.0) < 1e-12
    # TC3: all four root gains match the hand calculations.
    expected = [0.1636067182, 0.0610537837, 0.1368794314, 0.4083272214]
    observed = [split_by_attribute(DATASET_B, i)[1] for i in range(4)]
    assert all(abs(a - b) < 1e-8 for a, b in zip(observed, expected))
    # TC4: tree perfectly reproduces the 14 training records and rule predictions.
    tree = build_id3(DATASET_B, B_ATTRIBUTES, B_DOMAINS)
    assert all(predict_tree(tree, x) == y for _, x, y in DATASET_B)
    # TC5: fixed predictions on T from the full tree.
    expected_t = ["Yes", "Yes", "No", "Yes", "No", "Yes"]
```

```
    assert [predict_tree(tree, x) for _, x, _ in TEST_SET_T] == expected_t
    # TC6: contradictory labels force an empty version space; this must not crash.
    contradictory = DATASET_A + [("E6", DATASET_A[0][1], "No")]
    s6, g6, _ = candidate_elimination(contradictory, A_DOMAINS)
    assert len(enumerate_version_space(s6, g6, A_DOMAINS)) == 0
    # TC7: unseen Attendance value is handled by the parent majority fallback.
    unseen_prediction = predict_tree(tree, ("Very Low", "Irregular", "Passive", "Weak"))
    assert unseen_prediction == "Yes"
    # TC8: deterministic tie: Assignments precedes LMS at the Weak/High tie.
    weak_high = [r for r in DATASET_B if r[1][3] == "Weak" and r[1][0] == "High"]
    local_gains = [split_by_attribute(weak_high, i)[1] for i in (1, 2)]
    assert abs(local_gains[0] - local_gains[1]) < 1e-12
    assert tree.branches["Weak"].branches["High"].attribute == 1
    print("TC1–TC8: PASS")
    print("Dataset B root gains:", [round(v, 10) for v in observed])
    print("Dataset B training accuracy:", classification_metrics(DATASET_B, tree,
B_ATTRIBUTES)["accuracy"])
    print("Test Set T predictions:", expected_t)
    print("Test Set T metrics:", classification_metrics(TEST_SET_T, tree, B_ATTRIBUTES))
    print("Stratified 5-fold CV on B + T:", cross_validate(DATASET_B + TEST_SET_T, k=5))
def run_experiments():
    c_rows = generate_dataset_c()
    write_csv(c_rows, ROOT / "dataset_C_synthetic.csv")
    indices = list(range(len(c_rows)))
    random.Random(2026).shuffle(indices)
    train_indices, validation_indices = indices[:350], indices[350:]
    train_pool = [c_rows[i] for i in train_indices]
    validation = [c_rows[i] for i in validation_indices]
    # Learning curve: mean of 20 deterministic subsamples at each size.
    sizes = [4, 6, 8, 10, 12, 14, 25, 50, 100, 200, 350]
    train_scores, validation_scores = [], []
    for size in sizes:
        tr_scores, va_scores = [], []
```

```
    for seed in range(20):
        sample = random.Random(3000 + seed).sample(train_pool, size)
        model = build_id3(sample, B_ATTRIBUTES, B_DOMAINS)
        tr_scores.append(classification_metrics(sample, model, B_ATTRIBUTES)["accuracy"])
        va_scores.append(classification_metrics(validation, model, B_ATTRIBUTES)["accuracy"])
    train_scores.append(sum(tr_scores) / len(tr_scores))
    validation_scores.append(sum(va_scores) / len(va_scores))
# Complexity curve: maximum depth 0..4.
depths, tr_depth, va_depth, node_counts, leaf_counts = [], [], [], [], []
for depth in range(5):
    model = build_id3(train_pool, B_ATTRIBUTES, B_DOMAINS, max_depth=depth)
    tr_depth.append(classification_metrics(train_pool, model, B_ATTRIBUTES)["accuracy"])
    va_depth.append(classification_metrics(validation, model, B_ATTRIBUTES)["accuracy"])
    nodes, leaves, _ = tree_statistics(model)
    depths.append(depth); node_counts.append(nodes); leaf_counts.append(leaves)
# Noise experiment. Fixed independent seeds flip exactly 0%, 5%, and 10% of labels.
noise_rows = []
for noise in (0.0, 0.05, 0.10):
    flip_rng = random.Random(1000 + int(noise * 100))
    flip_indices = set(flip_rng.sample(range(500), round(500 * noise)))
    noisy = []
    for i, (name, instance, label) in enumerate(c_rows):
        new_label = ("No" if label == "Yes" else "Yes") if i in flip_indices else label
        noisy.append((name, instance, new_label))
    noisy_train = [noisy[i] for i in train_indices]
    noisy_validation = [noisy[i] for i in validation_indices]
    clean_validation = [c_rows[i] for i in validation_indices]
    models = {
        "Fully grown": build_id3(noisy_train, B_ATTRIBUTES, B_DOMAINS),
        "Controlled (depth 2, min split 10, min gain .01)": build_id3(
            noisy_train, B_ATTRIBUTES, B_DOMAINS, max_depth=2,
            min_samples_split=10, min_gain=0.01),
    }
```

```
    for label, model in models.items():
        n_nodes, n_leaves, depth = tree_statistics(model)
        noise_rows.append({
            "noise": noise, "model": label, "nodes": n_nodes, "leaves": n_leaves,
            "depth": depth,
            "training_accuracy_noisy": classification_metrics(noisy_train, model,
B_ATTRIBUTES)["accuracy"],
            "validation_accuracy_noisy": classification_metrics(noisy_validation, model,
B_ATTRIBUTES)["accuracy"],
            "validation_accuracy_clean": classification_metrics(clean_validation, model,
B_ATTRIBUTES)["accuracy"],
        })
  # Plots. Use matplotlib only; no external ML learner is invoked.
  try:
    import matplotlib.pyplot as plt
    plt.figure(figsize=(8.2, 4.5))
    plt.plot(sizes, train_scores, marker="o", color="#333333", label="Training accuracy")
    plt.plot(sizes, validation_scores, marker="s", color="#b36b00", label="Validation accuracy")
    plt.xlabel("Training records"); plt.ylabel("Accuracy")
    plt.title("Dataset C learning curve (20 repeated subsamples)")
    plt.ylim(0.4, 1.03); plt.grid(True, alpha=0.25); plt.legend()
    plt.tight_layout(); plt.savefig(FIG_DIR / "learning_curve.png", dpi=180); plt.close()
    plt.figure(figsize=(8.2, 4.5))
    plt.plot(depths, tr_depth, marker="o", color="#333333", label="Training accuracy")
    plt.plot(depths, va_depth, marker="s", color="#b36b00", label="Validation accuracy")
    plt.xlabel("Maximum tree depth"); plt.ylabel("Accuracy")
    plt.title("Model complexity on Dataset C")
    plt.xticks(depths); plt.ylim(0.4, 1.03); plt.grid(True, alpha=0.25); plt.legend()
    plt.tight_layout(); plt.savefig(FIG_DIR / "tree_complexity.png", dpi=180); plt.close()
    plt.figure(figsize=(8.2, 4.5))
    for label in ("Fully grown", "Controlled (depth 2, min split 10, min gain .01)"):
        subset = [r for r in noise_rows if r["model"] == label]
        plt.plot([r["noise"] * 100 for r in subset],
```

```
            [r["validation_accuracy_noisy"] for r in subset], marker="o",
            color=("#333333" if label == "Fully grown" else "#b36b00"), label=label)
    plt.xlabel("Injected label noise (%)"); plt.ylabel("Validation accuracy vs noisy labels")
    plt.title("Effect of label noise on Dataset C")
    plt.xticks([0, 5, 10]); plt.ylim(0.4, 1.03); plt.grid(True, alpha=0.25); plt.legend(fontsize=8)
    plt.tight_layout(); plt.savefig(FIG_DIR / "noise_effect.png", dpi=180); plt.close()
  except ImportError:
    print("Matplotlib is not installed; numerical experiment outputs are still available.")
  print("Dataset C balance:", Counter(y for _, _, y in c_rows))
  print("Dataset C unique combinations:", len(set(x for _, x, _ in c_rows)))
  print("Learning curve:", list(zip(sizes, train_scores, validation_scores)))
  print("Depth curve:", list(zip(depths, node_counts, leaf_counts, tr_depth, va_depth)))
  print("Noise results:", noise_rows)
if __name__ == "__main__":
  run_tests()
  run_experiments()
```